KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Laboratuvar Föyü

HAFTA 05

Laboratuvar Föyü

Yapıcı zinciri · kopya yapıcı ve derin kopya · statik fabrika metodu · AutoCloseable · başlatma sırası

ÖZET

Bu laboratuvar oturumunda önce doğrulaması dört yapıcıya kopyalanmış bir sınıfı **tek doğrulama noktalı** bir zincire dönüştürecek, ardından **derin kopya** yapan bir kopya yapıcı yazacak, yapıcısı `private` olan ve nesneyi yalnızca **statik fabrika metotlarıyla** üreten bir sınıf tasarlayacak, `AutoCloseable` gerçekleyen bir kaynak sınıfı yazıp `try-with-resources` ile sızıntısız kullanacak ve son olarak nesne **başlatma sırasını** deneysel olarak doğrulayacaksınız.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

Oturum Bilgileri

	Bilgi
Süre	120 dakika (2 ders saati)
Sorumlu	Arş. Gör. Betül MUMCU
Ortam	Visual Studio Code + JDK 21 (LTS)
Çalışma biçimi	Bireysel
Ön koşul	Hafta 5 ders notu Bölüm 3 ve 4 okunmuş olmalıdır
Teslim	Tüm `.java` dosyaları tek bir sıkıştırılmış klasör hâlinde, oturum bitiminde
Dosya adı biçimi	`Hafta05_<Numara>_<AdSoyad>.zip`
Değerlendirme	Görev 1–4 zorunlu (80 puan) · Görev 5 ileri düzey (20 puan)

Bu Bölümün Sonunda Öğrenci

- Aynı sınıfa birden çok yapıcı yazabilir ve `this(...)` ile zincirleyebilir.
- Doğrulama kurallarının tamamını **tek bir ana yapıcıda** toplayabilir; kuralı iki kez yazmaktan kaçınır.
- Kopya yapıcı yazabilir; değişebilir alanları çoğaltarak **derin kopya** üretebilir.
- Sığ kopyanın ürettiği paylaşım sorununu çalışan bir çıktı üzerinde gösterebilir.
- Yapıcıyı `private` yapıp nesne üretimini **statik fabrika metotlarına** taşıyabilir.
- `AutoCloseable` arayüzünü gerçekleyen bir sınıf yazabilir ve `try-with-resources` ile kaynak sızıntısını sayısal olarak önleyebilir.
- Nesne başlatma sırasını (static → örnek → yapıcı) çıktı üzerinde doğrulayabilir.



Şekil 5.1 — Laboratuvar oturumunun zaman planı

Çalışma Kuralları

1. Her `public` sınıf için **ayrı bir `.java` dosyası** oluşturun. Dosya adı, içindeki `public` sınıfın adıyla birebir aynı olmalıdır.

2. Kaynak dosyaları `src`, derlenmiş dosyaları `bin` klasöründe tutun: `javac -d bin src/*.java`
3. Tanımlayıcılarda (sınıf, metot, değişken adları) **Türkçe karakter kullanmayın**. Ekrana yazdırılan metinlerde de ASCII kalın.
4. Her sınıfta `toString()` metodunu ezin; sürücü programlar nesneleri bu metotla yazdırsın.
5. Doğrulama kuralını **yalnızca ana yapıcıda** yazın. Aynı kuralı iki yapıcıda gören değerlendirici puan keser.
6. Görevleri sırayla yapın; her görev bir öncekinin kavramını kullanır.

> **İNTİHAL UYARISI**
>
> Teslim edilen kod **bireysel** çalışmanın ürünü olmalıdır. Birbirinin kopyası olduğu belirlenen teslimlerin tamamı değerlendirme dışı bırakılır. Arkadaşınızla tartışabilirsiniz; kod alışverişi yapamazsınız.

Görev 0 — Ortam ve Ön Koşul Doğrulaması

Süre	Puan	Zorluk
10 dakika	Puansız — **zorunlu**	●○○

Amaç

Java ortamının çalıştığını doğrulamak ve `this(...)` zincirinin sözdizimini hatırlamak.

Görev Tanımı

`Gorev0_Kontrol` adında bir sınıf yazın. Aynı dosyada, `public` olmayan bir `Olcum` sınıfı tanımlayın. `Olcum` sınıfının üç yapıcısı olsun ve üçü de tek bir ana yapıcıda birleşsin.

Gereksinimler

1. `Olcum` sınıfının `ad` ve `deger` alanları `private final` olmalıdır.
2. `Olcum()` parametresiz yapıcısı `this("Adsiz", 0.0)` çağırmalıdır.
3. `Olcum(String, double)` **ana yapıcıdır**; ekrana `ANA yapici calisti` yazmalıdır.
4. `Olcum(Olcum kaynak)` kopya yapıcısı da `this(...)` ile ana yapıcıya devretmelidir.
5. `toString()` ezilmelidir.
6. `main` Java sürümünü yazdırmalı, üç yapıcıyı da denemeli ve kopyanın kaynakla **aynı nesne olmadığını** göstermelidir.

Beklenen Çıktı

```
▸ TERMINAL — java Gorev0_Kontrol
Java surumu : 25.0.2

--- Parametresiz yapici (zincirli) ---
  ANA yapici calisti
  Olcum[Adsiz = 0.00]

--- Iki parametreli ANA yapici ---
  ANA yapici calisti
  Olcum[Basinc = 1013.25]

--- Kopya yapici ---
  ANA yapici calisti
  Olcum[Basinc = 1013.25]
  Ayni nesne mi? false
```

> **İPUCU**
>
> Java sürümünü `System.getProperty("java.version")` ile okuyabilirsiniz. Sürüm numaranız farklı olabilir; bu bir sorun değildir. Çıktıda `ANA yapici calisti` satırının **üç kez** görünmesi, üç yapıcının da ana yapıcıya devrettiğinin kanıtıdır.

Kontrol Soruları

1. `ANA yapici calisti` satırı neden üç kez yazıldı? Kaç nesne üretildi?
2. Kopya yapıcının içine `this(kaynak.ad, kaynak.deger);` yerine `this.ad = kaynak.ad;` yazsaydınız ne değişirdi?

Görev 1 — Yapıcı Zinciri: Tek Doğrulama Noktası

Süre	Puan	Zorluk
25 dakika	20 puan	●○○

Amaç

Doğrulama kuralı her yapıcıya kopyalanmış bir sınıfı, kuralın **yalnızca bir yerde** bulunduğu bir zincire dönüştürmek.

Görev Tanımı

Aşağıdaki sorunlu `Kitap` sınıfı size verilmiştir. Bu sınıfı `this(...)` zinciriyle yeniden tasarlayın ve `Gorev1_KitapDemo` adında bir sürücü program yazarak davranışını gösterin.

Kitap.java (sorunlu sürüm)

```java
// SORUNLU SURUM - Gorev 1'de bunu yeniden tasarlayacaksiniz
public class Kitap {
  private final String isbn;
  private String baslik;
  private String yazar;
  private int sayfaSayisi;
  private int basimYili;

  public Kitap(String isbn, String baslik) {
    if (isbn == null || isbn.isBlank())  throw new IllegalArgumentException("ISBN bos");
    if (baslik == null || baslik.isBlank()) throw new IllegalArgumentException("Baslik bos");
    this.isbn = isbn;  this.baslik = baslik;
    this.yazar = "Bilinmiyor"; this.sayfaSayisi = 100; this.basimYili = 2026;
  }

```

```
16  public Kitap(String isbn, String baslik, String yazar){
17    if(isbn == null || isbn.isBlank())  throw new IllegalArgumentException("ISBN bos");
18    if(baslik == null || baslik.isBlank())throw new IllegalArgumentException("Baslik bos");
19    this.isbn = isbn;  this.baslik = baslik;
20     this.yazar = yazar;  this.sayfaSayisi = 100; this.basimYili = 2026;
21  }
22
23  public Kitap(String isbn, String baslik, String yazar, int sayfaSayisi, int basimYili){
24    if(isbn == null || isbn.isBlank())  throw new IllegalArgumentException("ISBN bos");
25    // baslik denetimi UNUTULDU
26    // sayfa sayisi denetimi UNUTULDU
27    // basim yili denetimi UNUTULDU
28    this.isbn = isbn;  this.baslik = baslik;  this.yazar = yazar;
29    this.sayfaSayisi = sayfaSayisi;  this.basimYili = basimYili;
30  }
31 }
```

Kod 5.1 — Yeniden tasarlanacak sınıf (verilen hâl)

Gereksinimler

Yapıcı	Beklenen davranış
`Kitap(isbn, baslik)`	`this(isbn, baslik, "Bilinmiyor")` çağırır
`Kitap(isbn, baslik, yazar)`	`this(isbn, baslik, yazar, 100)` çağırır
`Kitap(isbn, baslik, yazar, sayfa)`	`this(isbn, baslik, yazar, sayfa, 2026)` çağırır
`Kitap(isbn, baslik, yazar, sayfa, yil)`	**ANA YAPICI** — dört kuralın tamamı burada
`Kitap(Kitap kaynak)`	Kopya yapıcı; o da ana yapıcıya devreder
Doğrulama kuralları	ISBN boş olamaz · başlık boş olamaz · sayfa 1–5000 · yıl 1450–2026
Sabitler	`EN_AZ_SAYFA`, `EN_COK_SAYFA`, `BILINMIYOR` — `public static final`
Geçersiz değer	`IllegalArgumentException` fırlatılır; nesne **hiç doğmaz**
Türetilmiş bilgi	`getYas()` — `2026 - basimYili`, **alan tutulmaz**
`toString()`	`Kitap[978-975-01, "Baslik", Yazar, 100 s., 2026]`

Beklenen Çıktı

```
▸ TERMINAL — java Gorev1_KitapDemo (ilk bölüm)
=== 1) Dort yapici, dort ayri eksiklik duzeyi ===
  Kitap[978-975-01, "Nesne Yonelimli Programlama", Bilinmiyor, 100 s., 2026]
  Kitap[978-975-02, "Veri Yapilari", H. Yilmaz, 100 s., 2026]
  Kitap[978-975-03, "Algoritmalar", S. Demir, 720 s., 2026]
  Kitap[978-975-04, "Isletim Sistemleri", A. Kara, 640 s., 2019]

=== 2) Varsayilanlarin nereden geldigi ===
  k1 yazar       : Bilinmiyor   (2. yapicidan)
  k1 sayfa       : 100          (3. yapicidan)
```

```
  k1 basim yili  : 2026       (4. yapicidan)

=== 3) Dogrulama TEK yerde: ana yapici ===
  RET      : Sayfa sayisi gecersiz: 0
  RET      : ISBN bos olamaz
  RET      : Baslik bos olamaz
  RET      : Basim yili gecersiz: 2030
```

```
▸ TERMINAL — java Gorev1_KitapDemo (ikinci bölüm)
=== 4) Kopya yapici ===
  Kaynak : Kitap[978-975-04, "Isletim Sistemleri", A. Kara, 640 s., 2019]
  Kopya  : Kitap[978-975-04, "Isletim Sistemleri", A. Kara, 640 s., 2019]
  Ayni nesne mi? false
  --- kopyanin basligi degistirildi ---
  Kaynak : Isletim Sistemleri
  Kopya  : Isletim Sistemleri (2. baski)

=== 5) Turetilmis bilgi: getYas() ===
  Isletim Sistemleri -> 7 yasinda
```

DİKKAT

Doğrulama kurallarını **yalnızca ana yapıcıya** yazınız. Aynı denetimi iki yapıcıda gören değerlendirici, görev puanının yarısını keser. Bu görevin tamamı bu tek kuralın etrafında kuruludur.

İPUCU

`this(...)` çağrısı yapıcının **ilk satırı** olmak zorundadır. Zinciri kısadan uzuna kurun: 2 parametre → 3 → 4 → 5. Kopya yapıcı da zincirin bir parçasıdır; `this(kaynak.isbn, kaynak.baslik, ...)` yazınız.

Kontrol Soruları

1. Verilen sorunlu sürümde üçüncü yapıcıda hangi üç denetim unutulmuştu? Bu unutmanın **yapısal** nedeni nedir?
2. Sürücü programda `new Kitap("978-975-05", "Derleyiciler", "M. Ak", 0, 2020)` çağrısı hangi yapıcıdan ret aldı? Neden?
3. Varsayılan yazar değerini `"Bilinmiyor"` yerine `"Anonim"` yapmak isteseniz kaç satır değiştirirdiniz? Sorunlu sürümde kaç satır değiştirmek gerekirdi?

Görev 2 — Kopya Yapıcı ve Derin Kopya

Süre	Puan	Zorluk
25 dakika	20 puan	●●○

Amaç

Bir kopya yapıcının **gerçekten** kopya üretmesini sağlamak; sığ kopyanın ürettiği paylaşım sorununu çalışan bir çıktı üzerinde göstermek.

Görev Tanımı

`Personel` adında bir sınıf yazın. Sınıf hem ilkel türde alanlar hem de bir **dizi** alanı tutacaktır. İki kopya yapıcı yazacaksınız: biri bilerek **sığ**, diğeri **derin** kopya üretecek. Ardından `Gorev2_PersonelDemo` adında bir sürücü programla ikisi arasındaki farkı göstereceksiniz.

Gereksinimler

Öğe	Beklenen tasarım
`sicil`	`private final String` · yalnızca getter
`ad`	`private String` · getter + doğrulamalı setter
`maas`	`private double` · yalnızca getter; `zamYap(double yuzde)` ile değişir
`gorevler`	`private String[]` (8 elemanlı) · **değişebilir alan**
`gorevAdedi`	`private int` · dolu eleman sayısı
`Personel(sicil, ad)`	`this(sicil, ad, 30000.0)` çağırır
`Personel(sicil, ad, maas)`	**ANA YAPICI** — tüm doğrulama burada
`Personel(kaynak, sigKopya)`	`sigKopya true` ise diziyi **paylaştırır**, `false` ise `clone()` yapar
`Personel(kaynak)`	`this(kaynak, false)` — varsayılan **derin** kopya
`gorevEkle(String)`	Boş olamaz; dizi doluysa `RED:` yazar
`gorevDegistir(int, String)`	Sıra geçerli olmalı; yeni ad boş olamaz
`gorevleriYaz()`	Görevleri virgülle ayırıp döndürür (türetilmiş)
`toString()`	`Personel[P-1001, Betul Mumcu, 42000.00 TL, gorev={...}]`

Sürücü Programın Göstermesi Gerekenler

1. Derin kopyada kopyanın 0. görevini değiştirmek **kaynağı etkilemez**.
2. Sığ kopyada kopyanın 0. görevini değiştirmek **kaynağı da değiştirir**.
3. İlkel alanlar (`maas`) her iki durumda da kopyalanır.
4. Geçersiz istekler (`zamYap(-5)`, `gorevEkle("")`, `setAd("  ")`) reddedilir ve nesne durumunu korur.

Beklenen Çıktı

```
▸ TERMINAL — java Gorev2_PersonelDemo (ilk bölüm)
=== 1) Kaynak nesne hazirlaniyor ===
  Personel[P-1001, Betul Mumcu, 42000.00 TL, gorev={Laboratuvar, Odev degerlendirme}]

=== 2) DERIN kopya: diziler AYRI ===
  Kaynak : Personel[P-1001, Betul Mumcu, 42000.00 TL, gorev={Laboratuvar, Odev degerlendirme}]
  Kopya  : Personel[P-1001, Betul Mumcu (yedek kayit), 46200.00 TL, gorev={Laboratuvar (2. grup),
Odev degerlendirme}]
  Kopyanin 0. gorevi degisti, KAYNAGINKI DEGISMEDI.

=== 3) SIG kopya: dizi PAYLASILIYOR ===
  Kopyalama sonrasi:
    Kaynak : Teori dersi, Danismanlik
    Kopya  : Teori dersi, Danismanlik
  Kopyanin 0. gorevi degistirildi:
    Kaynak : UZAKTAN teori dersi, Danismanlik
    Kopya  : UZAKTAN teori dersi, Danismanlik
  KAYNAK DA DEGISTI -> iki nesne AYNI diziyi gosteriyor.
```

```
▸ TERMINAL — java Gorev2_PersonelDemo (ikinci bölüm)
=== 4) Ilkel alanlar her zaman kopyalanir ===
  a maas : 20000.00
  b maas : 30000.00
  double alan kopyalandi; sig/derin ayrimi ilkel turleri etkilemez.

=== 5) Gecersiz istekler ===
  RED: zam yuzdesi 0-100 araliginda olmali
  RED: gorev adi bos olamaz
  RED: ad bos olamaz
  Nesne durumu korundu : Personel[P-1003, Deneme Kisi, 30000.00 TL, gorev={(gorev yok)}]
```

SIK YAPILAN HATA

En sık yapılan hata, kopya yapıcının içinde `this.gorevler = kaynak.gorevler;` yazmaktır. Bu satır bir kopya **üretmez**; iki nesneyi aynı diziye bağlar. Metodun adının kopya yapıcı olması hiçbir şeyi değiştirmez — belirleyici olan koddur.

İPUCU

Farkı görebilmek için `gorevDegistir()` metodunu kullanın: `gorevEkle()` her nesnenin kendi `gorevAdedi` sayacını kullandığından farkı gizleyebilir. Var olan bir hücreyi değiştirmek paylaşımı doğrudan ortaya çıkarır.

Kontrol Soruları

1. Sığ kopyada `sig.gorevDegistir(0, "UZAKTAN teori dersi")` çağrısı neden kaynağın görev listesini de değiştirdi? Bellekte kaç dizi var?
2. `ad` alanı `String` türündedir ve kopya yapıcıda `clone()` çağrılmamıştır. Bu bir hata mıdır? Yanıtınızı gerekçelendirin.
3. `maas` alanı için sığ/derin ayrımı neden anlamsızdır?

Görev 3 — Statik Fabrika Metodu ve `private` Yapıcı

Süre	Puan	Zorluk
25 dakika	20 puan	●●○

Amaç

Yapıcının çözemediği bir sorunu fabrika metoduyla çözmek: aynı parametre listesiyle **farklı anlamlar** üretmek.

Görev Tanımı

`Bilet` adında bir sınıf yazın. Sınıf üç bilet türü üretmelidir: tam, öğrenci ve yaşlı. Üçü de `(String kalkis, String varis)` bilgisini aldığından **yapıcıyla yazılamazlar**; bu yüzden yapıcıyı `private` yapıp adlandırılmış fabrika metotları yazacaksınız.

Gereksinimler

Öğe	Beklenen tasarım
Sabitler	`TAM_UCRET = 400.0` · `OGRENCI_ORANI = 0.50` · `YASLI_ORANI = 0.30` (`public static final`)
Alanlar	`biletNo`, `kalkis`, `varis`, `tur`, `ucret` — hepsi `private final`
Yapıcı	`private` · dışarıdan `new Bilet(...)` yazılamaz
`biletNo`	`B-0001` biçiminde, sınıfın kendisi üretir (`private static int sonrakiSira`)
`tam(kalkis, varis)`	Tam ücret
`ogrenci(kalkis, varis)`	Tam ücretin %50'si
`yasli(kalkis, varis)`	Tam ücretin %70'i
`yasaGore(kalkis, varis, yas)`	≥65 → yaşlı · 18–25 → öğrenci · diğer → tam · geçersiz yaşta istisna
`uretilenBiletSayisi()`	`static` — o ana kadar üretilen bilet sayısı

`toString()`	`Bilet[B-0001, Trabzon->Of, TAM, 400.00 TL]`

Beklenen Çıktı

```
▸ TERMINAL — java Gorev3_BiletDemo (ilk bölüm)
=== 1) Adlandirilmis uretim yollari ===
  Bilet[B-0001, Trabzon->Of, TAM, 400.00 TL]
  Bilet[B-0002, Trabzon->Of, OGRENCI, 200.00 TL]
  Bilet[B-0003, Trabzon->Of, YASLI, 280.00 TL]
  Uc yapicinin da imzasi (String, String) olurdu -> DERLENMEZDI.

=== 2) Kural iceren fabrika: yasaGore ===
  yas= 12 -> Bilet[B-0004, Of->Rize, TAM, 400.00 TL]
  yas= 21 -> Bilet[B-0005, Of->Rize, OGRENCI, 200.00 TL]
  yas= 40 -> Bilet[B-0006, Of->Rize, TAM, 400.00 TL]
  yas= 70 -> Bilet[B-0007, Of->Rize, YASLI, 280.00 TL]

=== 3) Gecersiz istek ===
  RET : Gecersiz yas: -1
```

```
▸ TERMINAL — java Gorev3_BiletDemo (ikinci bölüm)
=== 4) Yapici disaridan cagrilamaz ===
  new Bilet("Of", "Rize", "TAM", 400.0);
  -> The constructor Bilet(String, String, String, double)
     is not visible   (derleme hatasi)

=== 5) Bilet numarasi sinifca uretilir ===
  Uretilen toplam bilet : 7
  Numaralar disaridan atanamaz; salt-okunur kimliktir.
```

> **DİKKAT**
>
> Ücretleri koda gömülü sayı olarak (`200.0`, `280.0`) yazmayınız. Sabitlerden hesaplayınız: `TAM_UCRET * (1 - OGRENCI_ORANI)`. İndirim oranı değişirse tek bir satır değişmelidir.

> **BİLGİ**
>
> `private static int sonrakiSira` bildirimindeki `static`, alanın nesnelere değil **sınıfa** ait olduğunu, yani tek kopya bulunduğunu söyler. Bu konunun ayrıntısı Hafta 6'nın işidir; bu görevde “tüm nesnelerin paylaştığı bir sayaç” olarak kullanmanız yeterlidir. Numara biçimi için `String.format("B-%04d", sonrakiSira++)` kullanınız.

Kontrol Soruları

1. `tam()`, `ogrenci()` ve `yasli()` metotlarını yapıcı olarak yazmayı deneyin. Derleyici hangi hatayı verir? Hatanın gerekçesi nedir?
2. Yapıcıyı `private` yapmasaydınız hangi güvence kaybolurdu? Sürücü programın hangi satırı derlenirdi?

3. `yasaGore()` metodu yeni bir nesne üretmek yerine `tam()`, `ogrenci()` ve `yasli()` metotlarını çağırıyor. Bu tercihin gerekçesi nedir?

Görev 4 — `AutoCloseable` ve `try-with-resources`

Süre	Puan	Zorluk
25 dakika	20 puan	●●○

Amaç

Java'da yıkıcı bulunmadığı için dış kaynakların nasıl güvenle kapatıldığını uygulamak ve kaynak sızıntısının olmadığını **sayısal olarak** kanıtlamak.

Görev Tanımı

`Gorev4_GunlukDemo` adında bir sürücü program ve aynı dosyada `AutoCloseable` gerçekleyen bir `GunlukDosyasi` sınıfı yazın. Sınıf, açılan ve kapanan kaynak sayısını `static` sayaçlarda tutmalıdır.

Gereksinimler

Öğe	Beklenen tasarım
Sınıf bildirimi	`class GunlukDosyasi implements AutoCloseable`
`acilan` / `kapanan`	`private static int` sayaçlar
Yapıcı	Dosya adı boş olamaz; `acilan++` ve `ACILDI` satırı
`kaydet(String)`	Kapalıysa `RED:` yazar; açıksa satır sayacını artırır
`close()`	Zaten kapalıysa sessizce döner (**idempotent**); aksi hâlde `kapanan++` ve `KAPANDI` satırı
`acilanSayisi()` / `kapananSayisi()`	`static` getter'lar
`toString()`	`GunlukDosyasi[hata.log, satir=1, acik=false]`

Sürücü Programın Göstermesi Gereken Altı Durum

1. Normal akış: blok bitince `close()` çağrılır.
2. Gövdede istisna fırlatılsa bile `close()` çağrılır — üstelik `catch` bloğundan **önce**.
3. Kapatılmış kaynağa yazma girişimi reddedilir.
4. İki kaynak birlikte açıldığında kapanma sırası **açılışın tersidir**.
5. `close()` iki kez çağrıldığında ikincisi etkisizdir.
6. Sonuçta açılan ve kapanan kaynak sayısı **eşittir** — sızıntı yoktur.

Beklenen Çıktı

```
▸ TERMINAL — java Gorev4_GunlukDemo (ilk bölüm)
=== 1) Normal akis: blok bitince close() ===
    ACILDI  : islem.log
    YAZILDI : islem.log [1] kullanici girisi
    YAZILDI : islem.log [2] rapor uretildi
    KAPANDI : islem.log (2 satir)

=== 2) Hata olsa bile close() cagrilir ===
    ACILDI  : hata.log
    YAZILDI : hata.log [1] islem basladi
    KAPANDI : hata.log (1 satir)
    HATA    : disk dolu
    Durum   : GunlukDosyasi[hata.log, satir=1, acik=false]

=== 3) Kapali kaynaga yazilamaz ===
    RED     : hata.log kapali, yazilamaz
```

```
▸ TERMINAL — java Gorev4_GunlukDemo (ikinci bölüm)
=== 4) Iki kaynak: kapanma sirasi TERSTIR ===
    ACILDI  : a.log
    ACILDI  : b.log
    YAZILDI : a.log [1] A satiri
    YAZILDI : b.log [1] B satiri
    KAPANDI : b.log (1 satir)
    KAPANDI : a.log (1 satir)

=== 5) close() birden cok kez cagrilirsa ===
    ACILDI  : tek.log
    KAPANDI : tek.log (0 satir)
    Sonuc   : ikinci close() etkisiz (idempotent)

=== 6) Toplam sayaclar ===
    Acilan kaynak sayisi   : 5
    Kapanan kaynak sayisi  : 5
    Fark 0 ise sizinti yok.
```

İPUCU

2. durumda `close()` çağrıldıktan sonra nesneye erişebilmek için referansı `try` bloğunun **dışında** tanımlanmış bir değişkende saklayın: `GunlukDosyasi disKopya = null;` sonra blok içinde `disKopya = g;` yazın.

DİKKAT

`try (...)` parantezinin içinde ürettiğiniz kaynak için ayrıca `close()` **yazmayınız**. Blok zaten çağırır; elle çağırmak iki kez kapatma denemesi olur. 5. durumda `close()`'u bilerek iki kez çağırdığımızı ve ikincisinin etkisiz kaldığını unutmayın.

Kontrol Soruları

1. Çıktının 2. bölümünde `KAPANDI` satırı `HATA` satırından önce yazıldı. Bu sıra neyi kanıtlar?
2. `close()` metodunu idempotent yazmak neden gereklidir? Idempotent olmasaydı 5. durumda ne olurdu?
3. Java'da yıkıcı olsaydı bu görev nasıl yazılırdı? Yıkıcının sağlayamayacağı hangi güvenceyi `try-with-resources` sağlıyor?

Görev 5 — İleri Düzey: Başlatma Sırası Deneyi (Bonus)

Süre	Puan	Zorluk
25 dakika	20 puan (bonus)	●●●

Amaç

Nesne başlatma sırasını ezberden değil **deneyle** öğrenmek: önce tahmin, sonra çalıştırma, sonra karşılaştırma.

Görev Tanımı

`Gorev5_BaslatmaSirasi` adında bir sürücü program ve aynı dosyada, her başlatma aşamasında ekrana bir satır yazan `Urun` adında bir sınıf yazın. Programı çalıştırmadan **önce** çıktıyı tahmin edip kâğıda yazın; sonra çalıştırıp karşılaştırın.

Gereksinimler

Aşama	Kod öğesi	Yazdırılacak iz
Sınıf düzeyi	`static int uretilen = izle(...)`	`[S1]`
Sınıf düzeyi	`static { izle(...); }`	`[S2]`
Nesne düzeyi	`int stok = izle(...)`	`[O1]`
Nesne düzeyi	`{ izle(...); }`	`[O2]`
Yapıcı	`Urun(String ad, int adet)` — **ana yapıcı**	`[Y1]`
Yapıcı	`Urun(String ad)` — `this(ad, 1)` sonrası	`[Y2]`
Yapıcı	`Urun()` — `this("Adsiz", 1)` sonrası	`[Y3]`

`izle(String mesaj, int deger)` adında `private static` bir yardımcı metot yazın; mesajı yazdırıp `deger` değerini döndürsün. Alan başlatıcılarında bu metodu çağırarak izi bırakabilirsiniz.

Sürücü Programın Yapması Gerekenler

1. `new Urun("Kalem")` — tek parametreli yapıcı.
2. `new Urun("Defter", 3)` — ana yapıcı doğrudan.
3. `new Urun()` — parametresiz yapıcı, `this(...)` zinciriyle.
4. Son olarak `Urun.uretilenSayisi()` değerini yazdırın.

Beklenen Çıktı

```
▸ TERMINAL — java Gorev5_BaslatmaSirasi (ilk bölüm)
--- Sinif henuz yuklenmedi ---

### 1. new Urun("Kalem") ###
      [S1] static alan baslaticisi
      [S2] static blok
      [O1] ornek alan baslaticisi (stok)
      [O2] ornek baslatma blogu
      [Y1] ANA yapici govdesi
      [Y2] tek parametreli yapici govdesi (this'ten SONRA)
  Sonuc: Urun[Kalem, adet=1, stok=5]

### 2. new Urun("Defter", 3) ###
      [O1] ornek alan baslaticisi (stok)
      [O2] ornek baslatma blogu
      [Y1] ANA yapici govdesi
  Sonuc: Urun[Defter, adet=3, stok=5]
```

```
▸ TERMINAL — java Gorev5_BaslatmaSirasi (ikinci bölüm)
### 3. new Urun() -> this(...) zinciri ###
      [O1] ornek alan baslaticisi (stok)
      [O2] ornek baslatma blogu
      [Y1] ANA yapici govdesi
      [Y3] parametresiz yapici govdesi (this'ten SONRA)
  Sonuc: Urun[Adsiz, adet=1, stok=5]

### 4. Sinif duzeyi sayac ###
  Uretilen urun sayisi : 3
  static bolum kac kez calisti? 1 (ilk kullanimda)
```

Teslim Edilecek Ek Belge

Bu görev için kodun yanında **yarım sayfalık bir rapor** teslim edin. Raporda şu üç soruya yanıt verin:

1. Tahmininiz ile gerçek çıktı hangi noktalarda ayrıldı?
2. `[S1]` ve `[S2]` neden yalnızca bir kez yazıldı?
3. 3. `new` çağrısında `[Y2]` neden hiç görünmedi?

> **İPUCU**
>
> Üçüncü sorunun yanıtı zincirin **yönündedir**: `Urun()` yapıcısı `this(ad, 1)` değil `this("Adsiz", 1)` çağırdığı için

doğrudan ana yapıcıya gider; tek parametreli yapıcı bu yolda hiç kullanılmaz. Ders notu Şekil 5.1'i yeniden inceleyiniz.

Kontrol Soruları

1. Örnek alan başlatıcısı ile örnek başlatma bloğunun çalışma sırasını belirleyen şey nedir?
2. `stok` alanının değeri her nesnede neden yeniden 5 oldu?
3. Bir `static` alan başlatıcısında henüz atanmamış bir örnek alanı okumaya çalışsanız ne olurdu?

Değerlendirme ve Teslim

Puanlama Ölçütleri

Görev	Ölçüt	Puan
Görev 1	Beş yapıcı ve zincir doğru (6) · doğrulama **yalnızca** ana yapıcıda (7) · sabitler tanımlı (3) · türetilmiş `getYas()` (4)	20
Görev 2	Yapıcı zinciri (4) · derin kopya `clone()` ile (7) · sığ kopya farkı çıktıda gösterilmiş (6) · doğrulamalı metotlar (3)	20
Görev 3	Yapıcı `private` (5) · üç fabrika metodu (6) · `yasaGore()` kuralları (5) · numara sınıfça üretiliyor (4)	20
Görev 4	`implements AutoCloseable` (4) · `close()` idempotent (5) · altı durumun tamamı gösterilmiş (7) · sayaçlar eşit (4)	20
Görev 5	Yedi izin tamamı (8) · çıktı beklenenle aynı (6) · rapor üç soruyu yanıtlıyor (6)	20 (bonus)
Toplam	Görev 1–4 zorunlu · Görev 5 bonus	**80 + 20**

Aşağıdaki durumlarda görev puanından kesinti uygulanır:

Durum	Kesinti
Aynı doğrulama kuralı birden çok yapıcıda yazılmış	Görev puanının %50'si
Kopya yapıcı yazılmış ama değişebilir alan kopyalanmamış	Görev puanının %40'ı
`try-with-resources` yerine elle `close()` çağrılmış	Görev puanının %40'ı
Yapıcı `public` bırakılmış (Görev 3)	Görev puanının %40'ı
`toString()` ezilmemiş	2 puan
Türkçe karakter içeren tanımlayıcı	3 puan
Dosya adı ile sınıf adı uyuşmuyor / derlenmiyor	Görev puanının tamamı

Teslim Öncesi Kontrol Listesi

- [ ] Tüm dosyalar hatasız derleniyor (`javac -d bin src/*.java` uyarı vermiyor).

Her sınıfta doğrulama kuralı **tek bir ana yapıcıda** toplandı.

Her kısa yapıcının ilk satırı `this(...)` çağrısı.

Kopya yapıcılarımda **değişebilir** her alan için `clone()` ya da eleman kopyası var.

`Bilet` sınıfının yapıcısı `private`; nesne yalnızca fabrika metotlarıyla üretiliyor.

Ücretler sabitlerden hesaplanıyor; koda gömülü sayı yok.

`GunlukDosyasi` sınıfım `AutoCloseable` gerçekliyor ve `close()` metodum idempotent.

Kaynaklarım `try (...)` parantezinin içinde üretiliyor; ayrıca elle `close()` çağırmıyorum.

Açılan ve kapanan kaynak sayısı çıktıda eşit görünüyor.

Dosya adları içindeki `public` sınıf adlarıyla birebir aynı; arşiv adı `Hafta05_<Numara>_<AdSoyad>.zip` biçiminde.

Sık Karşılaşılan Hatalar

Belirti	Olası neden	Çözüm
`Constructor call must be the first statement`	`this(...)` ilk satır değil	`this(...)` satırını en başa taşıyın
`Recursive constructor invocation`	Zincir kendine dönüyor	Zincirin ucunda `this(...)` içermeyen bir yapıcı bulunmalı
`The constructor Kitap() is undefined`	Parametresiz yapıcı yazılmadı	Ya açıkça yazın ya da o çağrıyı kullanmayın
`The constructor Bilet(...) is not visible`	Yapıcı `private`, dışarıdan çağrılıyor	Fabrika metodunu çağırın
`The blank final field ... may not have been initialized`	Bazı yapıcı yollarında `final` alan atanmıyor	Tüm yolların ana yapıcıya devrettiğinden emin olun
`does not implement AutoCloseable`	`try (...)` içine uygunsuz tür konmuş	Sınıfa `implements AutoCloseable` ekleyin
Kopya değiştirilince kaynak da değişiyor	Sığ kopya yapılmış	Değişebilir alan için `clone()` çağırın
Kapanan sayısı açılandan az	Bir kaynak `try (...)` dışında açılmış	Kaynağı parantez içine alın

> **BİLGİ**
>
> Laboratuvar sonunda çözümler paylaşılacaktır. Ancak çözümü görmeden önce görevleri kendiniz tamamlamanız, Hafta 6'daki `static` konusunu izleyebilmeniz için önemlidir: bu haftanın `sonrakiSira` sayacı ve fabrika metotları orada tam anlamını kazanacaktır.